

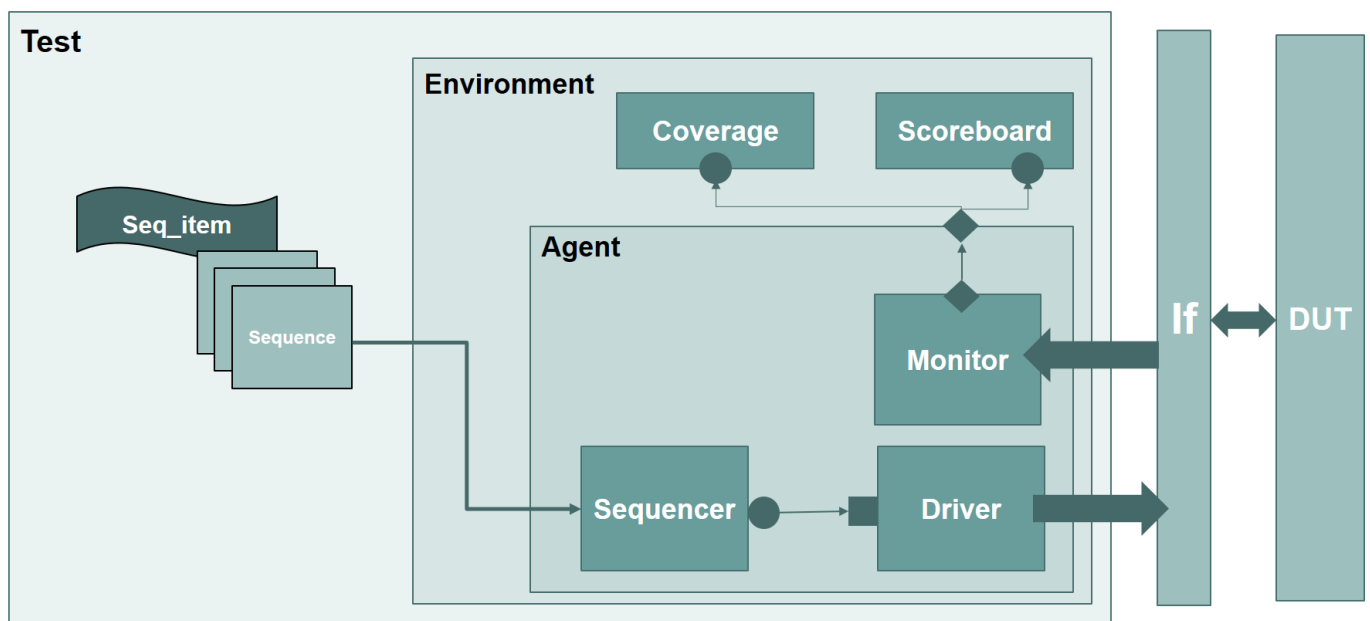
AXI4-Compliant Memory-Mapped Slave

Verification Using UVM

According to the previously provided specifications for AXI4 Compliant Memory design in the following Link: [AXI Specs](#)

Requirement

- It is required to build a full UVM Verification environment for the AXI-Memory Mapped design using all learned components and objects for the UVM and applying all concepts we walked through during the UVM learning phase.
- U can use the same design you used with SystemVerilog project after your fixes for the detected bugs in the previous project.
- For simplicity and to have a chance to go smoother with UVM you need to test the basic functionalities for the design of **READ** and **WRITE** operations without the **Burst** feature.
- U have to reach 100% Pass rate, 100% functional and code Coverage with giving acceptable justifications in case you miss the full percentage for the coverage.
- Practice Assertions with the built UVM Environment
- You have to implement full structure for the UVM and all components and objects you learned.
- U have to practice both the **Active** and the **Passive** Agent types. Beside the **AXI wrapper** agent which is the active one and carries its own checker and covergroups, u have to build a passive agent customized for the **Memory** itself with its own custom checker and its own custom covergroup for the memory behavior.
- Use the appropriate UVM Phasing and reporting with your environment.
- U have to draw the UVM Structure u used and built in your Verification process like below



- U have to practice the Factory Type Override on a component. Create a class **axi_driver_delay** which inherits from your driver and inserts random idle cycles between the transfers instead of driving them back to back. In a new test apply **set_type_override_by_type** inside the **build_phase** before calling **super.build_phase()** and run the same sequences without changing them. U have to deliver a snippet for **uvm_top.print_topology()** **before and after** the override showing that the instantiated type changed.

Extra Requirement (Optional)

- Verifying the **Brust** Feature for The AXI-Memory mapped design.
- Any other enhancements on the UVM structure you are building it is up to you and you are free to practice all the topics you learnt take the project as a chance for strengthening your knowledge.

Deliverables:

U have to deliver **two separated files**

- **Zip Folder** contains:
 - All the implemented uvm files and the **design files** and **run.do file**.
- **Separated PDF** contains:
 - Snippets for the implemented code.
 - Snippet for the waveform shows the different testcases clearly.
 - Snippets for the logs show the all printed values using UVM reporting.
 - Snippets for functional coverage report.
 - Snippets for code coverage report for all parameters [Line / toggle / branch /condition/...]
 - Snippets for Assertions Coverage report.
 - Snippet for Do file you have used for automating the process.

The delivered zip file must be named like **your_name_uvm_project.rar** for example:
Hassan_Khaled_uvm_project.rar also the PDF file **Hassan_Khaled_uvm_project.pdf**

Good Luck